

Lab 7 Control a relay - Virtual IoT Hardware

In this part of the lesson, you will add a relay to your virtual IoT device in addition to the soil moisture sensor, and control it based off the soil moisture level.

Virtual Hardware

The virtual IoT device will use a simulated Grove relay. This keeps this lab the same as using a Raspberry Pi with a physical Grove relay.

In a physical IoT device, the relay would be a normally-open relay (meaning the output circuit is open, or disconnected when there is no signal sent to the relay). A relay like this can handle output circuits up to 250V and 10A.

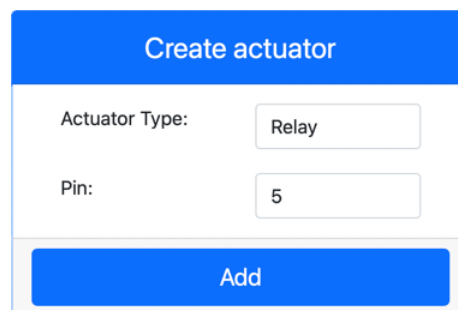
Add the relay to CounterFit

To use a virtual relay, you need to add it to the CounterFit app

Task

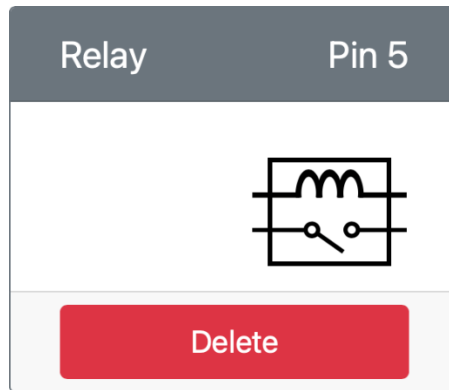
Add the relay to the CounterFit app.

1. Open the soil-moisture-sensor project from the last lesson in VS Code if it's not already open. You will be adding to this project.
2. Make sure the CounterFit web app is running
3. Create a relay:
 - i. In the *Create actuator* box in the *Actuators* pane, drop down the *Actuator type* box and select *Relay*.
 - ii. Set the *Pin* to 5
 - iii. Select the **Add** button to create the relay on Pin 5



Create actuator	
Actuator Type:	Relay
Pin:	5
Add	

The relay will be created and appear in the actuators list.



Program the relay

The soil moisture sensor app can now be programmed to use the virtual relay.

Task

Program the virtual device.

1. Open the soil-moisture-sensor project from the last lesson in VS Code if it's not already open. You will be adding to this project.
2. Add the following code to the app.py file below the existing imports:

```
from counterfit_shims.grove.grove_relay import GroveRelay
```

This statement imports the GroveRelay from the Grove Python shim libraries to interact with the virtual Grove relay.

3. Add the following code below the declaration of the ADC class to create a GroveRelay instance:

```
relay = GroveRelay(5)
```

This creates a relay using pin 5, the pin you connected the relay to.

4. To test the relay is working, add the following to the while True: loop:

```
relay.on()  
time.sleep(.5)  
relay.off()
```

The code turns the relay on, waits 0.5 seconds, then turns the relay off.

5. Run the Python app. The relay will turn on and off every 10 seconds, with a half second delay between turning on and off. You will see the virtual relay in the CounterFit app close and open as the relay is turned on and off.



Control the relay from soil moisture

Now that the relay is working, it can be controlled in response to soil moisture readings.

Task

Control the relay.

1. Delete the 3 lines of code that you added to test the relay. Replace them with the following code in its place:

```
if soil_moisture > 450:
    print("Soil Moisture is too low, turning relay on.")
    relay.on()
else:
    print("Soil Moisture is ok, turning relay off.")
    relay.off()
```

This code checks the soil moisture level from the soil moisture sensor. If it is above 450, it turns on the relay, turning it off if it goes below 450.

🧑 Remember the capacitive soil moisture sensor reads the lower the soil moisture level, the more moisture there is in the soil and vice versa.

2. Run the Python app. You will see the relay turn on or off depending on the soil moisture levels. Change the *Value* or the *Random* settings for the soil moisture sensor to see the value change.

Soil Moisture: 638

Soil Moisture is too low, turning relay on.

Soil Moisture: 452

Soil Moisture is too low, turning relay on.

Soil Moisture: 347

Soil Moisture is ok, turning relay off.

👤 Your code should look like this:

```
#Your name Here

from counterfit_connection import CounterFitConnection
CounterFitConnection.init('127.0.0.1', 5000)
import time
from counterfit_shims_grove.adc import ADC
from counterfit_shims_grove.grove_relay import GroveRelay

adc = ADC()
relay = GroveRelay(5)

while True:
    soil_moisture = adc.read(0)
    print("Soil moisture:", soil_moisture)

    time.sleep(10)
    if soil_moisture > 450:
        print("Soil Moisture is too low, turning relay on.")
        relay.on()
    else:
        print("Soil Moisture is ok, turning relay off.")
        relay.off()
```

😊 Your virtual soil moisture sensor controlling a relay program was a success! Take a screenshot of VSCode with the app running and save it as 1.jpg

Control your plant over MQTT

So far your relay is controlled by the IoT device directly based off a single soil moisture reading. In a commercial irrigation system, the control logic will be centralized, allowing it to make decisions on watering using data from multiple sensors, and allowing any configuration to be changed in one single place. To simulate this, you can control the relay over MQTT.

Task - control the relay over MQTT

1. Add the relevant MQTT libraries/pip packages and code to your soil-moisture-sensor project to connect to MQTT. Name the client ID as soilmoisturesensor_client prefixed by your ID.

⚠️ You can refer to the instructions for connecting to MQTT in project 1, lesson 4 if needed.

2. Add the relevant device code to send telemetry with the soil moisture settings. For the telemetry message, name the property `soil_moisture`.

⚠ You can refer to the instructions for sending telemetry to MQTT in project 1, lesson 4 if needed.

3. Create some local server code to subscribe to telemetry and send a command to control the relay in a folder called `soil-moisture-sensor-server`. Name the property in the command message `relay_on`, and set the client ID as `soilmoisturesensor_server` prefixed by your ID. Keep the same structure as the server code you wrote for project 1, lesson 4 as you will be adding to this code later in this lesson.

⚠ You can refer to the instructions for sending telemetry to MQTT and sending commands over MQTT in project 1, lesson 4 if needed.

4. Add the relevant device code to control the relay from received commands, using the `relay_on` property from the message. Send `true` for `relay_on` if the `soil_moisture` is greater than 450, otherwise send `false`, the same as the logic you added for the IoT device earlier.

⚠ You can refer to the instructions for responding to commands from MQTT in project 1, lesson 4 if needed.

Make sure the code is running on your device and local server, and test it out by changing soil moisture levels, either by changing the values sent by the virtual sensor, or by changing the moisture levels of the soil by adding water or removing the sensor from the soil.

Add timing to your plant control server

The server code can be modified to add control around the timing of the watering cycle, and waiting for the soil moisture levels to change. The server logic for controlling the relay timing is:

1. Telemetry message received
2. Check the soil moisture level
3. If it's ok, do nothing. If the reading is too high (meaning the soil moisture is too low) then:
 - i. Send a command to turn the relay on
 - ii. Wait for 5 seconds

- iii. Send a command to turn the relay off
- iv. Wait for 20 seconds for the soil moisture levels to stabilize

The watering cycle, the process from receiving the telemetry message to being ready to process soil moisture levels again, takes about 25 seconds. We're sending soil moisture levels every 10 seconds, so there is an overlap where a message is received whilst the server is waiting for soil moisture levels to stabilize, which could start another watering cycle.

There are two options to work around this:

- Change the IoT device code to only send telemetry every minute, this way the watering cycle will be completed before the next message is sent
- Unsubscribe from the telemetry during the watering cycle

The first option is not always a good solution for large farms. The farmer might want to capture the soil moisture levels as the soil is being watered for later analysis, for example to be aware of water flow in different areas on the farm to guide more targeted watering. The second option is better - the code is just ignoring telemetry when it can't use it, but the telemetry is still there for other services that might subscribe to it.

 IoT data is not sent from only one device to only one service, instead many devices can send data to a broker, and many services can listen to the data off the broker. For example, one service could listen to soil moisture data and store it in a database for analysis at a later date. Another service can also listen to the same telemetry to control an irrigation system.

Task - add timing to your plant control server

Update your server code to run the relay for 5 seconds, then wait 20 seconds.

1. Open the soil-moisture-sensor-server folder in VS Code if it isn't already open. Make sure the virtual environment is activated.
2. Open the app.py file
3. Add the following code to the app.py file below the existing imports:

```
import threading
```

This statement imports threading from Python libraries, threading allows python to execute other code while waiting.

4. Add the following code before the handle_telemetry function that handles telemetry messages received by the server code:

```
water_time = 5
```

```
wait_time = 20
```

This defines how long to run the relay for (water_time), and how long to wait afterwards to check the soil moisture (wait_time).

5. Below this code, add the following:

```
def send_relay_command(client, state):  
    command = { 'relay_on' : state }  
    print("Sending message:", command)  
    client.publish(server_command_topic, json.dumps(command))
```

This code defines a function called send_relay_command that sends a command over MQTT to control the relay. The telemetry is created as a dictionary, then converted to a JSON string. The value passed into state determines if the relay should be on or off.

6. After the send_relay_code function, add the following code:

```
def control_relay(client):  
    print("Unsubscribing from telemetry")  
    mqtt_client.unsubscribe(client_telemetry_topic)  
  
    send_relay_command(client, True)  
    time.sleep(water_time)  
    send_relay_command(client, False)  
  
    time.sleep(wait_time)  
  
    print("Subscribing to telemetry")  
    mqtt_client.subscribe(client_telemetry_topic)
```

This defines a function to control the relay based off the required timing. It starts by unsubscribing from telemetry so that soil moisture messages are not processed whilst the watering is happening. Next it sends a command to turn the relay on. It then waits for the water_time before sending a command to turn the relay off. Finally it waits for the soil moisture levels to stabilize for wait_time seconds. It then re-subscribes to telemetry.

7. Change the handle_telemetry function to the following:

```
def handle_telemetry(client, userdata, message):
```

```
def handle_telemetry(client, userdata, message):  
  
    payload = json.loads(message.payload.decode())  
    print("Message received:", payload)  
  
    if payload['soil_moisture'] > 450:  
        threading.Thread(target=control_relay, args=(client,)).start()
```

This code checks the soil moisture level. If it is greater than 450, the soil needs watering, so it calls the control_relay function. This function is run on a separate thread, running in the background.

8. Make sure your IoT device is running, then run this code. Change the soil moisture levels and observe what happens to the relay - it should turn on for 5 seconds then remain off for at least 20 seconds, only turning on if the soil moisture levels are not sufficient.

```
(.venv) → soil-moisture-sensor-server$ python app.py
```

```
Message received: {'soil_moisture': 457}
```

```
Unsubscribing from telemetry
```

```
Sending message: {'relay_on': True}
```

```
Sending message: {'relay_on': False}
```

```
Subscribing to telemetry
```

```
Message received: {'soil_moisture': 302}
```

A good way to test this in a simulated irrigation system is to use dry soil, then pour water in manually whilst the relay is on, stopping pouring when the relay turns off.

 If you want to use a pump to build a real irrigation system, then you can use a [6V water pump](#) with a [USB terminal power supply](#). Make sure the power to or from the pump is connected via the relay.

Take a screenshot of VSCode with the soil-moisture-sensor app running and save it as 2.jpg

Take a screenshot of VSCode with the soil-moisture-sensor-server app running and save it as 3.jpg

Upload all 3 screenshots to the assignment on moodle.